

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I Gurusurya G 192311332 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Gurusurya

Annexure A

Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.
2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.
3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.
4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.
5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Implement and Compare MD5 and SHA-256 in Python

Code:

```
import hashlib
import time

messages = [
    "Hello World",
    "Cryptography",
    "Cloud Computing",
    "Network Security",
    "Hello World"
]

print("MD5 and SHA-256 Hash Comparison")
print("-" * 70)

for message in messages:
    md5_hash = hashlib.md5(message.encode()).hexdigest()
    sha256_hash = hashlib.sha256(message.encode()).hexdigest()

    print("\nMessage:", message)
    print("MD5      :", md5_hash)
    print("SHA-256 :", sha256_hash)

# Performance comparison
test_message = "This is a performance testing message." * 1000
iterations = 10000

start = time.perf_counter()

for _ in range(iterations):
    hashlib.md5(test_message.encode()).hexdigest()

md5_time = time.perf_counter() - start

start = time.perf_counter()

for _ in range(iterations):
    hashlib.sha256(test_message.encode()).hexdigest()

sha256_time = time.perf_counter() - start
```

```

print("\nPerformance Comparison")
print("MD5 Time   :", md5_time, "seconds")
print("SHA-256 Time:", sha256_time, "seconds")

```

Output:

The screenshot shows a code editor with a file named 'main.py' and an 'Output' pane. The code in 'main.py' performs a performance comparison between MD5 and SHA-256 hashing for three different messages. The 'Output' pane displays the results of the script execution.

```

main.py
10 for message in messages:
22
23 # Performance comparison
24 test_message = "This is a performance testing message." * 1000
25 iterations = 10000
26
27 start = time.perf_counter()
28
29 for _ in range(iterations):
30     hashlib.md5(test_message.encode()).hexdigest()
31
32 md5_time = time.perf_counter() - start
33
34 start = time.perf_counter()
35
36 for _ in range(iterations):
37     hashlib.sha256(test_message.encode()).hexdigest()
38
39 sha256_time = time.perf_counter() - start
40
41 print("\nPerformance Comparison")
42 print("MD5 Time   :", md5_time, "seconds")

```

Output

```

MD5 and SHA-256 Hash Comparison
-----

Message: Hello World
MD5      : b10a8db164e0754105b7a99be72e3fe5
SHA-256  : a591a6d40bf420404a011733cfb7b190d62c65bf0bcd32b57b277d9
          ad9f146e

Message: Cryptography
MD5      : 64ef07ce3e4b420c334227eecb3b3f4c
SHA-256  : b584eec728548aced5a66c0267dd520a00871b5e7b735b2d8202f867
          19f61857

Message: Cloud Computing
MD5      : 3de3a4a550829dc4376329c29771bf78
SHA-256  : 1c5d6a56ec257e5fe6f733e7e81f6f2571475d44c09faa9ecd9aa2ff1
          c4a49ecd

Message: Network Security
MD5      : e9609b040993009e0e36b9a4d7161b87
SHA-256  : f95b4782c97bd99b554584e864216f33b5bd97f1d85e61fa9bc1b3f0

```

2. RSA Digital Signature in Python

Code:

```
import hashlib
```

```
p = 61
```

```
q = 53
```

```
n = p * q
```

```
phi = (p - 1) * (q - 1)
```

```
e = 17
```

```
d = 2753
```

```
message = "This is a confidential message."
```

```

hash_value = int.from_bytes(
    hashlib.sha256(message.encode()).digest(),

```

```
        byteorder="big"
    )

    hash_value = hash_value % n

    signature = pow(hash_value, d, n)

    print("Original Message:")
    print(message)

    print("\nSHA-256 Hash (reduced for demonstration):")
    print(hash_value)

    print("\nDigital Signature:")
    print(signature)

    received_hash = int.from_bytes(
        hashlib.sha256(message.encode()).digest(),
        byteorder="big"
    ) % n

    verified_hash = pow(signature, e, n)

    print("\nVerification Result:")

    if received_hash == verified_hash:
        print("Signature Verified Successfully")
    else:
        print("Signature Verification Failed")

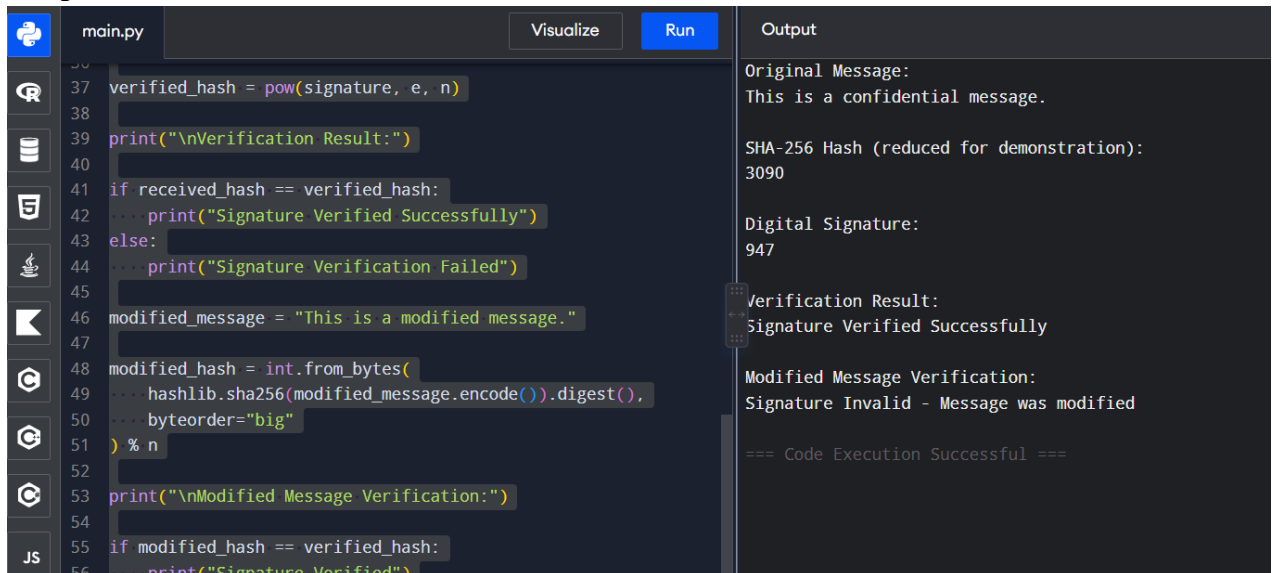
    modified_message = "This is a modified message."

    modified_hash = int.from_bytes(
        hashlib.sha256(modified_message.encode()).digest(),
        byteorder="big"
    ) % n

    print("\nModified Message Verification:")

    if modified_hash == verified_hash:
        print("Signature Verified")
    else:
        print("Signature Invalid - Message was modified")
```

Output:



The screenshot shows a code editor with a file named `main.py`. The code implements a digital signature verification process. It calculates a SHA-256 hash for a message, compares it with a received hash, and verifies a digital signature. The output panel on the right shows the results of these operations.

```
37 verified_hash = pow(signature, e, n)
38
39 print("\nVerification Result:")
40
41 if received_hash == verified_hash:
42     print("Signature Verified Successfully")
43 else:
44     print("Signature Verification Failed")
45
46 modified_message = "This is a modified message."
47
48 modified_hash = int.from_bytes(
49     hashlib.sha256(modified_message.encode()).digest(),
50     byteorder="big"
51 ) % n
52
53 print("\nModified Message Verification:")
54
55 if modified_hash == verified_hash:
56     print("Signature Verified")
```

Output:

```
Original Message:
This is a confidential message.

SHA-256 Hash (reduced for demonstration):
3090

Digital Signature:
947

Verification Result:
Signature Verified Successfully

Modified Message Verification:
Signature Invalid - Message was modified

=== Code Execution Successful ===
```

3. HMAC Implementation in Python

Code:

```
import hashlib
import hmac

message = b"Important confidential message"
secret_key = b"MySecretKey123"

simple_hash = hashlib.sha256(message).hexdigest()

hmac_value = hmac.new(
    secret_key,
    message,
    hashlib.sha256
).hexdigest()

print("Original Message:")
print(message.decode())

print("\nSimple SHA-256 Hash:")
print(simple_hash)

print("\nHMAC-SHA256:")
```

```

print(hmac_value)

received_message = b"Important confidential message"

calculated_hmac = hmac.new(
    secret_key,
    received_message,
    hashlib.sha256
).hexdigest()

if hmac.compare_digest(hmac_value, calculated_hmac):
    print("\nHMAC Verification: Successful")
else:
    print("\nHMAC Verification: Failed")

modified_message = b"Modified confidential message"

modified_hmac = hmac.new(
    secret_key,
    modified_message,
    hashlib.sha256
).hexdigest()

if hmac.compare_digest(hmac_value, modified_hmac):
    print("Modified message accepted")
else:
    print("Modified message rejected")

```

Output:

The screenshot shows a code editor with a file named `main.py` and a terminal window displaying the output of the code. The code in `main.py` imports `hashlib` and `hmac`, defines a message and a secret key, calculates a simple SHA-256 hash and an HMAC, and then verifies the HMAC. The terminal output shows the original message, the simple SHA-256 hash, the HMAC, and the verification results for both the original and modified messages.

```

main.py Visualize Run
1 import hashlib
2 import hmac
3
4 message = b"Important confidential message"
5 secret_key = b"MySecretKey123"
6
7 simple_hash = hashlib.sha256(message).hexdigest()
8
9 hmac_value = hmac.new(
10     secret_key,
11     message,
12     hashlib.sha256
13 ).hexdigest()
14
15 print("Original Message:")
16 print(message.decode())
17
18 print("\nSimple SHA-256 Hash:")
19 print(simple_hash)
20
21 print("\nHMAC-SHA256:")
22 print(hmac_value)

```

Output

```

Original Message:
Important confidential message

Simple SHA-256 Hash:
86ff7a2a79310937ae1e537ee7300015aab91da5272245143c38d14a25e28953

HMAC-SHA256:
b14c768aa47696c234ded41df7aa031be551672fe3907218cbd5c9c354a6cbce

HMAC Verification: Successful
Modified message rejected

=== Code Execution Successful ===

```


4. Simplified Kerberos Authentication in Python

Code:

```
import time
import hmac
import hashlib
CLIENT_KEY = b"client-secret"
SERVER_KEY = b"server-secret"
def create_ticket(username, server, timestamp, lifetime=30):
    data = f'{username}|{server}|{timestamp}|{lifetime}'.encode()

    ticket_mac = hmac.new(
        SERVER_KEY,
        data,
        hashlib.sha256
    ).hexdigest()

    return {
        "username": username,
        "server": server,
        "timestamp": timestamp,
        "lifetime": lifetime,
        "mac": ticket_mac
    }

def validate_ticket(ticket):
    current_time = int(time.time())

    data = (
        f'{ticket["username"]}|'
        f'{ticket["server"]}|'
        f'{ticket["timestamp"]}|'
        f'{ticket["lifetime"]}'
    ).encode()

    expected_mac = hmac.new(
        SERVER_KEY,
        data,
        hashlib.sha256
    ).hexdigest()

    valid_mac = hmac.compare_digest(
        ticket["mac"],
```

```

        expected_mac
    )

    not_expired = (
        current_time <=
        ticket["timestamp"] + ticket["lifetime"]
    )

    return valid_mac and not_expired

username = "Alice"
server = "FileServer"

timestamp = int(time.time())

ticket = create_ticket(
    username,
    server,
    timestamp
)

print("Ticket Generated:")
print(ticket)

if validate_ticket(ticket):
    print("\nTicket Validation: SUCCESS")
    print("Client authenticated successfully.")
else:
    print("\nTicket Validation: FAILED")

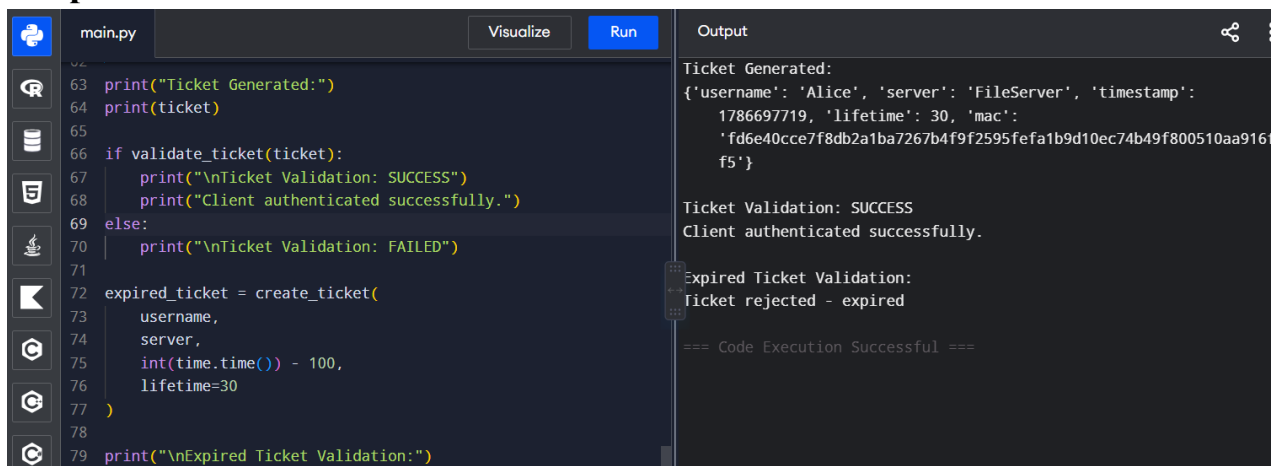
expired_ticket = create_ticket(
    username,
    server,
    int(time.time()) - 100,
    lifetime=30
)

print("\nExpired Ticket Validation:")

if validate_ticket(expired_ticket):
    print("Ticket accepted")
else:
    print("Ticket rejected - expired")

```

Output:



The screenshot shows a Python IDE with a file named 'main.py'. The code in the editor includes a function to generate a ticket, a function to validate a ticket, and a function to create an expired ticket. The output window on the right shows the results of the code execution: a ticket is generated for 'Alice' with a lifetime of 30, the ticket is validated successfully, and an expired ticket is rejected.

```
63 print("Ticket Generated:")
64 print(ticket)
65
66 if validate_ticket(ticket):
67     print("\nTicket Validation: SUCCESS")
68     print("Client authenticated successfully.")
69 else:
70     print("\nTicket Validation: FAILED")
71
72 expired_ticket = create_ticket(
73     username,
74     server,
75     int(time.time()) - 100,
76     lifetime=30
77 )
78
79 print("\nExpired Ticket Validation:")
```

Output:

```
Ticket Generated:
{'username': 'Alice', 'server': 'FileServer', 'timestamp':
 1786697719, 'lifetime': 30, 'mac':
 'fd6e40cce7f8db2a1ba7267b4f9f2595fefa1b9d10ec74b49f800510aa916
 f5'}
```

Ticket Validation: SUCCESS
Client authenticated successfully.

Expired Ticket Validation:
Ticket rejected - expired

=== Code Execution Successful ===

5. ElGamal Digital Signature Scheme in Python

Code:

```
import hashlib
p = 467
g = 2
x = 127
y = pow(g, x, p)

message = "Digital signature using ElGamal"
h = int.from_bytes(
    hashlib.sha256(message.encode()).digest(),
    byteorder="big"
)
k = 3
r = pow(g, k, p)
def mod_inverse(a, m):
    return pow(a, -1, m)
q = p - 1

s = ((h - x * r) * mod_inverse(k, q)) % q

print("Public Key:")
print("(p, g, y) =", (p, g, y))

print("\nPrivate Key:")
print(x)
```

```

print("\nMessage:")
print(message)

print("\nSignature:")
print("(r, s) =", (r, s))
left = pow(g, h, p)
right = (pow(y, r, p) * pow(r, s, p)) % p

print("\nVerification:")

if left == right:
    print("Signature Verified")
else:
    print("Signature Invalid")

```

Output:

main.py	Visualize	Run	Output
<pre> 17 18 s = ((h - x * r) * mod_inverse(k, q)) % q 19 20 print("Public Key:") 21 print("(p, g, y) =", (p, g, y)) 22 23 print("\nPrivate Key:") 24 print(x) 25 26 print("\nMessage:") 27 print(message) 28 29 print("\nSignature:") 30 print("(r, s) =", (r, s)) 31 left = pow(g, h, p) 32 right = (pow(y, r, p) * pow(r, s, p)) % p 33 34 print("\nVerification:") 35 36 if left == right: 37 print("Signature Verified") 38 else: </pre>			<pre> Public Key: (p, g, y) = (467, 2, 132) Private Key: 127 Message: Digital signature using ElGamal Signature: (r, s) = (8, 234) Verification: Signature Verified === Code Execution Successful === </pre>